

# Onion Quality Grading

## AI-Powered Batch Inspection System

---

System Architecture & Technical Overview

Prepared for presentation / review

Detect → Classify → Grade → Price → Report

# 1. What This System Does

A farmer, trader, or procurement inspector photographs a batch of onions -- a single onion, a handful, or a full crate -- using a phone. The system detects every individual onion in the photo, classifies each one as **healthy**, **damaged**, **rotten**, or **sprouted**, aggregates the results into a quality percentage, applies a configurable grading rule to assign **Grade 1**, **Grade 2**, or **URS** (Unfit for Sale), estimates a price per quintal, and produces an auditable digital report -- replacing manual, inconsistent visual inspection.

The design deliberately separates **what the AI is responsible for** from **what plain business logic is responsible for**:

| AI Model                                                                                        | Business Rules (plain Python)                                                      |
|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Find each onion in the photo and classify its condition (healthy / damaged / rotten / sprouted) | Turn those classifications into a grade, a price, and a pass/fail confidence check |
| Learned from training photos -- improves only with better/more training data                    | Fully deterministic and instantly configurable -- no retraining needed             |

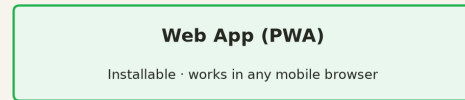
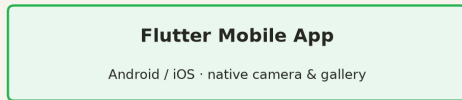
This matters because it means the grading thresholds and prices can be changed by an administrator in seconds (no data science required), while the AI model can be improved independently as better training photos become available.

## 2. System Architecture

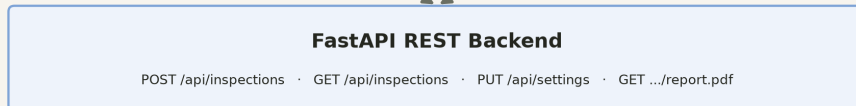
### Onion Quality Grading — System Architecture

*AI-powered onion batch inspection: detect → classify → grade → price → report*

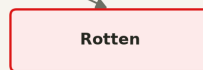
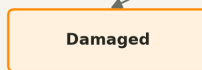
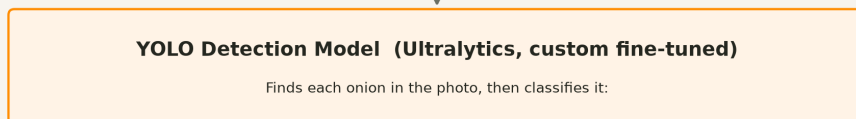
① CLIENT LAYER — camera-first entry, no menu in the way



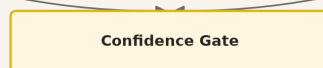
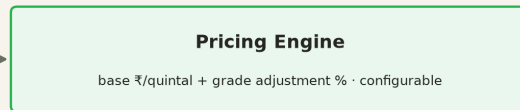
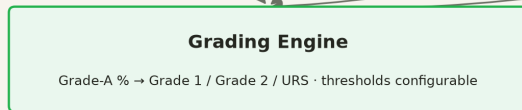
② API LAYER



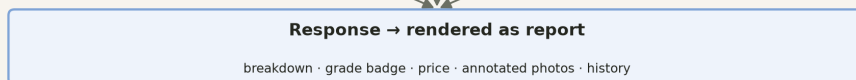
③ AI LAYER — the only part that's a trained model; everything else is plain code



④ BUSINESS LOGIC LAYER — plain, auditable Python rules, deliberately NOT another model



⑤ DATA LAYER



*back to Client Layer (①)*

## 3. Component Breakdown

### Client Apps (Flutter + Web PWA)

Two independent clients talking to the same backend API. Both open directly onto a camera-first screen -- no dashboard or menu between opening the app and taking a photo. One photo analyzes immediately as a single inspection; two or more photos are treated as one batch inspection automatically, with no mode to choose upfront.

*Technology:* Flutter (Dart) for native Android/iOS | vanilla HTML/CSS/JS PWA for the web, installable to a home screen

### FastAPI REST Backend

Stateless HTTP API that receives uploaded photos, invokes the AI model, runs the grading and pricing engines, persists the result, and serves history and PDF reports back to either client.

*Technology:* Python, FastAPI, Uvicorn

### YOLO Detection Model

A custom-trained Ultralytics YOLO model, fine-tuned specifically on onion photographs. For every photo it draws a bounding box around each onion it finds and assigns one of four classes with a confidence score. This is the only machine-learned component in the system -- everything downstream of it is deterministic code.

*Technology:* Ultralytics YOLO (PyTorch)

### Grading Engine

Computes Grade-A % (the healthy share of the batch) and applies configurable thresholds: Grade 1 if Grade-A % is at or above the Grade-1 threshold, Grade 2 if it clears the lower threshold, otherwise URS. Thresholds are stored in settings and editable from the app -- no retraining or redeployment needed to change a cutoff.

*Technology:* Plain Python, threshold-driven

### Pricing Engine

Applies a configurable percentage adjustment per grade on top of a base market price to produce an estimated Rs./quintal figure. Explicitly labeled in the UI and PDF as a demo/configurable estimate, not an official procurement rate.

*Technology:* Plain Python, config-driven

### Confidence Gate

If the average detection confidence for an inspection falls below a configurable threshold, the result is flagged "Low confidence -- manual verification recommended" instead of being silently presented as certain. This keeps the system honest about when the AI is unsure.

*Technology:* Plain Python

### Data Layer

Every inspection's counts, grade, price, and settings snapshot are persisted for history and audit. Original and annotated (bounding-box-drawn) images are stored per inspection. A full PDF report can be generated and downloaded for any inspection.

*Technology:* SQLite | local file storage | ReportLab (PDF generation)

## 4. Request Lifecycle -- What Happens When a Photo Is Submitted

| # | Step                     | What happens                                                                                                                                                                            |
|---|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Capture</b>           | User opens the app and taps Take Photo (or Gallery). One photo can be taken, or several for a batch -- no mode selection needed.                                                        |
| 2 | <b>Upload</b>            | Photo(s) are sent as a single multipart POST to /api/inspections, along with an optional batch label and an optional pixel-per-cm calibration value.                                    |
| 3 | <b>Detect + Classify</b> | The YOLO model runs on each photo, returning a bounding box, a class (healthy/damaged/rotten/sprouted), and a confidence score for every onion found.                                   |
| 4 | <b>Aggregate</b>         | Per-image results are summed into batch-wide totals: total onions detected and a count per class, across all submitted photos.                                                          |
| 5 | <b>Grade</b>             | Grade-A % = healthy ÷ total. The grading engine compares this against the configured thresholds to assign Grade 1, Grade 2, or URS.                                                     |
| 6 | <b>Price</b>             | The pricing engine applies the grade's configured adjustment to the base market price to produce an estimated Rs./quintal.                                                              |
| 7 | <b>Confidence check</b>  | If average detection confidence is below the configured threshold, the result is flagged for manual review.                                                                             |
| 8 | <b>Persist</b>           | The inspection record, original photos, and annotated (boxed) photos are saved. A unique inspection ID is generated.                                                                    |
| 9 | <b>Respond</b>           | The client renders the full breakdown, grade badge, confidence pill, estimated price, and annotated photos -- and can download a PDF report or browse this inspection later in history. |

## 5. Technology Stack

| Layer             | Technology                    | Why                                                            |
|-------------------|-------------------------------|----------------------------------------------------------------|
| Mobile client     | Flutter (Dart)                | Single codebase for Android and iOS, native camera access      |
| Web client        | HTML / CSS / vanilla JS (PWA) | No build step, installable, runs in any mobile browser         |
| Backend API       | FastAPI (Python) + Uvicorn    | Fast, typed, auto-documented REST API                          |
| AI / detection    | Ultralytics YOLO (PyTorch)    | State-of-the-art real-time object detection, fine-tunable      |
| Grading & pricing | Plain Python (rule-based)     | Auditable, instantly configurable, no ML infrastructure needed |
| Database          | SQLite                        | Zero-ops embedded database, sufficient for inspection history  |
| Report generation | ReportLab                     | Server-side PDF generation with embedded photos                |

## 6. Current Model Status & Roadmap

The detection/classification model is trained and improving iteratively. As of the current deployed version:

| Class    | Status                                                     |
|----------|------------------------------------------------------------|
| Healthy  | Strong -- high precision and recall                        |
| Damaged  | Strong -- high precision and recall                        |
| Rotten   | Working, moderate recall -- improving with each retrain    |
| Sprouted | Not yet reliable -- root cause identified, fix in progress |

The system is architected so that improving the model is a drop-in change: the backend automatically loads whichever trained model file is newest, with no code changes required to ship an improved version. Two known, well-understood gaps remain on the roadmap:

- **Sprouted/rotten classification** -- traced to a labeling issue in the training data pipeline (now corrected); awaiting a clean retrain on the fixed dataset.
- **Very dense, heavily-overlapping crate photos** -- the model reliably finds onions in normally-spaced photos; extremely packed piles need additional real-world training photos at that density to fully resolve.

All grading thresholds and pricing figures are explicitly labeled throughout the app and PDF reports as configurable prototype values, not official procurement rates, until validated against an authorized specification.